Git Commit Best Practices

1 Comment / By Dot Net Tutorials / February 22, 2026

Back to: Microservices using ASP.NET Core Web API Tutorials

Git Commit Best Practices

A **commit** is not just a button you click in Git. It is a **record of progress** in your project. Every commit tells a story about what changed and why. In real projects, commits are used daily to review work, track bugs, roll back mistakes, and understand how the code evolved over time.

As a fresher, learning **good commitment habits early** is extremely important. Bad commit practices (huge commits, unclear messages, broken code) create confusion and make teamwork difficult. Good commit practices make you look professional and make projects easy to maintain.

What Should a Commit Represent?

A commit should represent **one meaningful logical unit of work**. That **logical unit of work** can be:

- Adding a small feature
- Fixing one bug
- Refactoring a specific part of the code
- Updating configuration or validation

A commit should answer this question clearly: **What exactly did I change in this commit?** If you cannot explain it in one sentence, the commit is probably doing too much.

Examples of good commits:

- Add TeacherController GET endpoint
- Fix the null reference in StudentService
- Added logging in a controller
- Updated validation rules for DTO
- Configure JSON options to disable camelCase

Bad commit:

- Added teacher API + fixed login bug + updated UI + changed DB scripts

Why? Because later, if one part causes issues, you can't easily isolate and rollback. A good commit should make sense **on its own**. If someone reads your commit later, they should understand what was changed without needing to guess.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Why are commits important in real projects?

Commits matter because real projects are team projects and long-running projects. In real-world projects where multiple members are working, commits are used to:

- Review code during pull requests
- Identify when and where a bug was introduced
- Roll back to a stable version
- Understand project history, i.e., who changed what and why
- Collaborate with multiple developers safely

Without good commits:

- Debugging becomes hard
- Rollback becomes risky
- Code reviews become painful
- Team productivity drops

Commits are the **backbone of collaboration**.

What Is a Commit Message?

A **commit message** is a short explanation of what the commit does. It tells:

- What changed
- Why was it changed

Think of it like a label on a parcel:

- If the label is clear → Easy to identify later
- If the label is unclear → You'll regret it later

Bad Commit Messages:

- Update Code
- Changes Done
- Fix
- Final

Good Commit Messages:

- Add TeacherController GET endpoint
- Fix the student not found handling in StudentController
- Fix the null reference in Student Service

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Add CORS policy to allow all origins (dev only)
- Add logging in Student Controller

Commit messages are used daily by:

- Team leads during review
- Developers while debugging
- DevOps while deploying
- You (after 2 months), when you forget what you did

A commit message is the readable label that explains what your commit does. So, a good commit message helps **future you** and **other developers** understand the change instantly.

How to Write a Good Commit Message?

A good commit message should be:

- Clear
- Specific
- Short
- Meaningful

Simple rules for writing a good commit message:

- Start with an action verb (Add, Fix, Update, Remove, Refactor)
- Mention what you changed
- Optionally mention why, if needed

Good Examples:

- Add Teacher API controller
- Fix the student not found handling in StudentController
- Disable camelCase JSON naming to match model properties
- Add CORS policy AllowAll for frontend integration
- Add JWT authentication middleware
- Refactor repository methods for async usage

Avoid:

- Done
- Trial
- Changes
- Working now

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- Final

Why are small commits better than large commits?

Small commits are easier to manage and safer.

Benefits of small commits:

- Easier to review (reviewers don't miss issues)
- Easier to debug (you can pinpoint the exact commit that broke things)
- Easier to rollback (remove only the bad change, keep the good ones)
- Lower chance of merge conflicts

Large commits:

- Mix multiple changes together
- It's hard to understand
- Make debugging difficult
- Increase chances of mistakes
- Bigger conflicts
- Painful rollbacks

Example:

Instead of one huge commit:

- Added Authentication + Caching + Logging + UI Changes

Prefer smaller commits:

- Add authentication setup
- Add login endpoint
- Add token generation logic
- Add the authorization attribute to endpoints

Rule: Commit frequently, but commit meaningful changes. Small commits reduce risk and make code easier to review and fix.

What is an atomic commit?

An **atomic commit** means:

- One commit = one logical change
- The commit is complete and correct
- The commit does not break the build

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- The commit does not contain half the work

Example of atomic commits:

- Commit 1: Add DTO for Teacher
- Commit 2: Add TeacherController GET endpoint
- Commit 3: Register TeacherService in DI

Each commit builds safely, step-by-step. Atomic commits make history clean and reliable.

When should we commit code?

Commit when your change is:

- Meaningful
- Working
- Testable
- Not breaking build

Best moments to commit:

- After completing a small feature step
- After fixing one bug completely
- After refactoring a small area safely
- After adding a test and ensuring it passes

Avoid committing:

- Midway experiments
- Broken compilation
- Incomplete features

A Good Habit: Finish one logical step → Run build/tests → Commit. Do **not** wait until the end of the day to commit everything together. Commit **step by step** as progress happens.

Why broken or half-done code should not be committed?

Because commits are treated as trusted checkpoints. If you commit broken code:

- The branch becomes unstable
- Teammates pulling your branch get build failures
- CI/CD pipelines fail
- Debugging becomes harder because history has unsafe points

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Rule: **If it doesn't build / tests fail, don't commit.**

What is staging, and why does it matter?

Staging means selecting which changes to include in the next commit.

- You can commit only the relevant files
- You avoid accidentally committing extra changes
- You keep commits clean and atomic

Example:
You fixed a bug in StudentService.cs, but also changed formatting in Program.cs.

- Stage only StudentService.cs
- Commit the bug fix separately
- Keep formatting changes for later (or discard)

This is how professionals keep commit history clean.

How to review changes before committing in Visual Studio?

Before you commit, always review what is going on.

Step-by-step in Visual Studio

- View → Git Changes
- You will see Files Changed
- Click each file to see the diff view (before vs after)
- Stage only the correct files
- Write a clear commit message
- Commit

What to check in review:

- Did I accidentally change unrelated files?
- Any secrets added?
- Any debug code left? (Console logs, hardcoded values)
- Does it match the commit message?

Conclusion:

Commit best practices are simple: commit **small, meaningful, working changes** with a clear message, and use **staging** to include only what belongs in that commit. Avoid committing secrets, generated files, or half-done/broken

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

code. When you follow this habit, your project history stays clean, and it becomes much easier to review changes, debug issues, and roll back safely.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent | Do Not Sell My Information

Explore our developer-friendly HTML to PDF API | Printed using PDFCrowd | HTML to PDF